

CryptoMacro Dashboard

How the Website Was Built From Scratch — Technical Construction Guide

Document generated 2026-08-11 12:54 UTC

1. Purpose and product scope

CryptoMacro is a client-side cryptocurrency and macro financial dashboard. It was designed as a 2026-ready SPA (single-page application) that works without a custom backend server: market data is pulled from public REST and WebSocket providers, portfolio state lives in the browser, and the site is published for free on GitHub Pages.

Primary product modules: live asset charts (crypto, equity, aggregates, pairs), Crypto Fear & Greed (line + pie), multi-duration timeframes, Live Portfolio Tracker (USD/EUR), Shemitah cycle intelligence, and a multi-tier data failover layer.

2. Technology stack

Layer	Technology	Why chosen
UI framework	React 19 + TypeScript	Component model, type safety, wide ecosystem
Build tool	Vite 8	Fast HMR, simple production bundling
Styling	Tailwind CSS v4	Utility-first design tokens, dark theme
Charts	TradingView lightweight-charts	Performant canvas candles / lines
Server state	TanStack Query	Caching, retries, stale-while-revalidate
Offline cache	IndexedDB (idb)	Persist candles & portfolio offline
Live prices	Binance WebSocket	Low-latency miniTicker stream
Hosting	GitHub Pages + Actions	Free static hosting with CI deploy
E2E tests	Playwright (Chromium)	UI + API regression suite

3. Project bootstrap (from zero)

3.1 Scaffold

The application started as a Vite React-TypeScript project. Core commands:

```
npm create vite@latest CryptoDashboard -- --template react-ts
cd CryptoDashboard
npm install
npm install @tanstack/react-query lightweight-charts idb clsx
npm install -D tailwindcss @tailwindcss/vite @playwright/test
```

3.2 Folder architecture

```
src/
  components/  UI widgets (charts, nav, portfolio, shemitah)
  constants/   Asset registry, timeframe maps, Shemitah data
  context/     Live WebSocket price provider
  hooks/       React Query + portfolio hooks
  services/    financialApi, cache, fxRates, portfolioStore
  types/       Shared domain TypeScript types
```

e2e/	Playwright API + UI tests
scripts/	Build-time SPX prefetch for GitHub Pages
public/data/	Static SPX snapshot used as production backup

4. Build steps — chronological feature construction

Step A — Design tokens & shell layout

Defined dark fintech theme in `src/index.css` (Plus Jakarta Sans + IBM Plex Mono). Built **Dashboard**, **Sidebar**, **MobileNav**, sticky header and market ticker strip.

Step B — Asset registry

Created **ASSETS** in `constants/assets.ts` for BTC/ETH/SOL/ADA, SPX, BTC.D/TOTAL/TOTAL2/TOTAL3, BTC pairs, and Fear & Greed. Timeframes expanded to Hours 1–8, Days 1–7, Weeks 1–4, Months 1–12, Years 1–4, ALL.

Step C — Multi-tier financial API layer

`services/financialApi.ts` normalizes all providers to `Candle[]`. Crypto/pairs: Binance REST (+ direct race) → CryptoCompare. Aggregates: shared Binance BTC/ETH base + CoinLore/Paprika global race. Equity: Yahoo multi-host/SPY → CORS proxies (prod) → build-time SPX snapshot. Fear & Greed: alternative.me history.

Step D — Interactive charts

InteractiveChart uses lightweight-charts (candles or line). **TimeframeSelector** is a two-step unit+count control. Shemitah bands/events overlay on BTC/SPX when toggled.

Step E — Live ticks

LiveTicksContext opens one Binance combined `miniTicker` `WebSocket` for all `binanceSymbol` assets and fans prices into cards and the portfolio valuation engine.

Step F — Portfolio tracker

`LocalStorage` + `IndexedDB` portfolio with amount held, optional avg buy price, USD/EUR FX (Frankfurter/ECB + CoinGecko backup), allocation bar/donut, live P&L. No login. Export/import UI intentionally removed per product request; persistence remains automatic.

Step G — Security hardening

CSP meta, referrer policy, nosniff. Portfolio JSON allow-list for asset ids, finite amount bounds, prototype pollution rejection. `WebSocket` messages validated (symbol allow-list, finite price). Shemitah overlay clears DOM without dangerous HTML injection of untrusted markup.

Step H — Free production deploy (GitHub only)

GitHub Actions workflow builds with `GITHUB_PAGES=true` (base path `/CryptoDashboard/`), prefetches SPX snapshot, uploads Pages artifact. Live URL: <https://trifmarius1.github.io/CryptoDashboard/>

Step I — Automated QA

Playwright suite under `e2e/` covers market APIs, navigation, charts, portfolio actions, security meta tags, and unique DOM ids. Run with `npm test`.

5. Data flow (runtime)

Browser UI

- TanStack Query ■■ fetchCandles / fetchQuote / overview
- ■■ memory cache (90s)
- ■■ live REST (Binance / Yahoo / CoinLore / F&G / FX)
- ■■ IndexedDB candle cache (failover)
- ■■ synthetic series (last resort)
- Websocket ■■ LiveTicksContext ■■ card prices + portfolio
- LocalStorage/IDB ■■ portfolio holdings

6. Local development & production commands

```
npm install
npm run dev          # http://127.0.0.1:5173 with Vite API proxies
npm run prefetch:spx # refresh public/data/spx.json
npm run build        # prefetch + tsc + vite build
npm test             # Playwright E2E + API suite
git push origin master # triggers GitHub Pages deploy
```

7. External APIs used

Provider	Use	Access path
Binance REST + WS	Crypto/pair OHLCV + live ticks	Proxy in dev; direct in prod
Yahoo Finance	S&P 500 / SPY	Proxy in dev; multi-host + snapshot backup
CoinLore / CoinPaprika	Global mcap & dominance	Proxy / direct
alternative.me	Fear & Greed history	Proxy / direct
Frankfurter (ECB)	USD→EUR FX	Proxy / direct
CoinGecko	Optional global/FX backup	Rate-limited; not chart hot path

8. Known production constraints

- GitHub Pages is static: Vite **/api/*** proxies exist only in local dev.
- Yahoo often blocks browser CORS on Pages; build-time **spx.json** guarantees SPX charts.
- Third-party free CORS proxies are best-effort only and may rate-limit.
- Public market APIs can throttle under heavy traffic; caches mitigate short outages.
- This product is educational / informational — not financial advice.

9. Repository

Source: <https://github.com/trifmarius1/CryptoDashboard>

Live site: <https://trifmarius1.github.io/CryptoDashboard/>